

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 18
MESIN ABSTRAK



DISUSUN OLEH:
Rayhan Ahza Widyamukti
109082500210
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Mesin abstrak adalah model komputasi yang mendeskripsikan sekumpulan operasi dasar terhadap suatu struktur data tertentu, tanpa menjelaskan detail bagaimana data tersebut sebenarnya disimpan atau diolah di balik layar. Pengguna mesin hanya perlu mengetahui kontrak dari setiap operasi—apa yang menjadi masukan dan apa yang dihasilkan—tanpa perlu mengetahui representasi internalnya. Pendekatan ini erat kaitannya dengan konsep abstraksi data dan enkapsulasi dalam ilmu komputer, di mana suatu tipe data didefinisikan berdasarkan kumpulan operasi yang bisa dilakukan terhadapnya, bukan berdasarkan struktur penyimpanannya. Dengan cara ini, perubahan pada implementasi internal tidak akan memengaruhi program lain yang menggunakan mesin tersebut, selama kontrak operasinya tetap sama.

Secara umum, sebuah mesin abstrak terdiri dari dua bagian utama, yaitu keadaan (state) dan operasi dasar (primitive operations). Keadaan adalah representasi data yang dikelola oleh mesin dan bersifat tersembunyi dari pengguna, sedangkan operasi dasar adalah satu-satunya cara yang disediakan untuk membaca atau mengubah keadaan tersebut. Operasi ini umumnya dibagi menjadi dua jenis, yakni prosedur yang mengubah keadaan mesin tanpa mengembalikan nilai, dan fungsi yang mengembalikan suatu nilai berdasarkan keadaan mesin saat itu. Setiap kali sebuah operasi dipanggil, keadaan mesin akan berubah dari satu kondisi ke kondisi berikutnya sesuai aturan yang telah ditetapkan, sehingga seluruh algoritma yang lebih kompleks dapat dibangun hanya dari kombinasi operasi-operasi dasar tersebut.

Pada soal-soal di atas, konsep mesin abstrak diterapkan pada dua kasus yang berbeda. Pada mesin abstrak domino, keadaannya berupa tumpukan 28 kartu domino beserta jumlah kartu yang tersisa, sedangkan operasinya berupa kocokKartu, ambilKartu, gambarKartu, dan nilaiKartu yang menjadi satu-satunya cara mengakses dan memanipulasi kartu tersebut, sehingga algoritma permainan seperti Gappleh, galiKartu, dan sepasangKartu dapat dibangun tanpa perlu mengetahui detail penyimpanan array di baliknya. Sementara itu, pada mesin abstrak karakter, keadaannya berupa untaian karakter dan posisi pembacaan saat ini, dengan operasi dasar start, maju, eop, dan cc yang menyembunyikan detail pengindeksan string—mirip dengan konsep iterator atau cursor dalam ilmu komputer—sehingga algoritma pembacaan seperti penghitungan jumlah karakter, huruf "A", dan pasangan "LE" dapat disusun murni dari keempat operasi tersebut.

SOAL LATIHAN

Statement perulangan

1. Mesin Dasar Domino

Source Code:

```
// Rayhan Ahza Widyamukti_109082500210
package main

import "fmt"

type Domino struct {
    suitA int
    suitB int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu    [28]Domino
    sisaKartu int
}

func kocokKartu(d *Dominoes) {
    var indeks int
    indeks = 0
    var i int
    for i = 0; i <= 6; i++ {
        var j int
        for j = i; j <= 6; j++ {
            d.kartu[indeks].suitA = i
            d.kartu[indeks].suitB = j
            d.kartu[indeks].nilai = i + j
            if i == j {
                d.kartu[indeks].balak = true
            } else {
                d.kartu[indeks].balak = false
            }
            indeks = indeks + 1
        }
    }
    d.sisaKartu = 28

    var kunciAcak int
    kunciAcak = 5
    var k int
    for k = 0; k < 28; k++ {
        var tukar int
        tukar = (k * kunciAcak) % 28
        var temp Domino
        temp = d.kartu[k]
        d.kartu[k] = d.kartu[tukar]
        d.kartu[tukar] = temp
    }
}
```

```

    }
}

func ambilKartu(d *Dominoes) Domino {
    var kartuTerambil Domino
    if d.sisaKartu > 0 {
        kartuTerambil = d.kartu[d.sisaKartu-1]
        d.sisaKartu = d.sisaKartu - 1
    }
    return kartuTerambil
}

func gambarKartu(kartu Domino, suit int) int {
    var hasil int
    if suit == 0 {
        hasil = kartu.suitA
    } else {
        hasil = kartu.suitB
    }
    return hasil
}

func nilaiKartu(kartu Domino) int {
    var hasil int
    hasil = kartu.nilai
    return hasil
}

func main() {
    fmt.Println(" SOAL 1 - MESIN ABSTRAK DOMINO")
    fmt.Println(" Disusun oleh: Rayhan Ahza Widyamukti_109082500210")

    var himpunanDomino Dominoes
    kocokKartu(&himpunanDomino)

    fmt.Println("\nKartu domino telah dikocok. Total kartu:",
himpunanDomino.sisaKartu)

    var jumlahAmbil int
    fmt.Print("\nMasukkan jumlah kartu yang ingin diambil
dari tumpukan: ")
    fmt.Scan(&jumlahAmbil)

    var i int
    for i = 1; i <= jumlahAmbil; i++ {
        if himpunanDomino.sisaKartu <= 0 {
            fmt.Println("Kartu pada tumpukan sudah
habis.")
            break
        }
        var kartuAmbil Domino
        kartuAmbil = ambilKartu(&himpunanDomino)

        var suitSisiA int
        var suitSisiB int

```

```

        suitSisiA = gambarKartu(kartuAmbil, 0)
        suitSisiB = gambarKartu(kartuAmbil, 1)

        var totalNilai int
        totalNilai = nilaiKartu(kartuAmbil)

        fmt.Println("\nKartu ke-", i, "berhasil diambil:")
        fmt.Println("  Suit sisi A :", suitSisiA)
        fmt.Println("  Suit sisi B :", suitSisiB)
        fmt.Println("  Nilai kartu :", totalNilai)
        fmt.Println("  Balak?      :", kartuAmbil.balak)
        fmt.Println("  Sisa kartu pada tumpukan:",
himpunanDomino.sisaKartu)
    }

    fmt.Println("\nProgram selesai dijalankan oleh Rayhan
Ahza Widyamukti_109082500210")
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 18> go run dasarDomino.go
  Sisa kartu pada tumpukan: 26

Kartu ke- 3 berhasil diambil:
  Suit sisi A : 2
  Suit sisi B : 6
  Nilai kartu : 8
  Balak?      : false
  Sisa kartu pada tumpukan: 25

Program selesai dijalankan oleh Rayhan Ahza Widyamukti_109082500210

```

Deskripsi Program:

Program ini mendefinisikan struct Domino (suit sisi A, suit sisi B, nilai, dan status balak) serta Dominoes (array 28 kartu plus penghitung sisa kartu), lalu mengimplementasikan empat operasi dasar mesin: kocokKartu yang membangun ke-28 kartu domino unik (0,0 sampai 6,6) dan mengacak posisinya menggunakan rumus modulo (karena tidak boleh pakai library math/rand), ambilKartu yang mengambil kartu teratas tumpukan sambil mengurangi sisa kartu, gambarKartu yang membaca salah satu suit dari sebuah kartu, dan nilaiKartu yang mengembalikan total nilai kartu. Program meminta input jumlah kartu yang ingin diambil pengguna, lalu menampilkan detail setiap kartu yang berhasil diambil.

2. Realisasi Operasi Mesin Domino

Source Code:

```
// Rayhan Ahza Widyamukti_109082500210
package main

import "fmt"

type Domino struct {
    suitA int
    suitB int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu    [28]Domino
    sisaKartu int
}

func kocokKartu(d *Dominoes) {
    var indeks int
    indeks = 0
    var i int
    for i = 0; i <= 6; i++ {
        var j int
        for j = i; j <= 6; j++ {
            d.kartu[indeks].suitA = i
            d.kartu[indeks].suitB = j
            d.kartu[indeks].nilai = i + j
            if i == j {
                d.kartu[indeks].balak = true
            } else {
                d.kartu[indeks].balak = false
            }
            indeks = indeks + 1
        }
    }
    d.sisaKartu = 28

    var kunciAcak int
    kunciAcak = 5
    var k int
    for k = 0; k < 28; k++ {
        var tukar int
        tukar = (k * kunciAcak) % 28
        var temp Domino
        temp = d.kartu[k]
        d.kartu[k] = d.kartu[tukar]
        d.kartu[tukar] = temp
    }
}

func ambilKartu(d *Dominoes) Domino {
```

```

        var kartuTerambil Domino
        if d.sisaKartu > 0 {
            kartuTerambil = d.kartu[d.sisaKartu-1]
            d.sisaKartu = d.sisaKartu - 1
        }
        return kartuTerambil
    }

func gambarKartu(kartu Domino, suit int) int {
    var hasil int
    if suit == 0 {
        hasil = kartu.suitA
    } else {
        hasil = kartu.suitB
    }
    return hasil
}

func nilaiKartu(kartu Domino) int {
    var hasil int
    hasil = kartu.nilai
    return hasil
}

func galiKartu(d *Dominoes, pembanding Domino, hasilGalian
*Domino) bool {
    var ditemukan bool
    ditemukan = false

    for d.sisaKartu > 0 {
        var kartuSaatIni Domino
        kartuSaatIni = ambilKartu(d)

        var suitA1 int
        var suitB1 int
        suitA1 = gambarKartu(kartuSaatIni, 0)
        suitB1 = gambarKartu(kartuSaatIni, 1)

        var suitA2 int
        var suitB2 int
        suitA2 = gambarKartu(pembanding, 0)
        suitB2 = gambarKartu(pembanding, 1)

        if suitA1 == suitA2 || suitA1 == suitB2 || suitB1 ==
suitA2 || suitB1 == suitB2 {
            *hasilGalian = kartuSaatIni
            ditemukan = true
            break
        }
    }
    return ditemukan
}

func sepasangKartu(kartu1 Domino, kartu2 Domino) bool {
    var total int
    total = nilaiKartu(kartu1) + nilaiKartu(kartu2)
}

```

```

        var hasil bool
        if total == 12 {
            hasil = true
        } else {
            hasil = false
        }
        return hasil
    }

func main() {
    fmt.Println(" SOAL 2 - GALIKARTU & SEPASANGKARTU")
    fmt.Println(" Disusun oleh: Rayhan Ahza
Widyamukti_109082500210")

    var himpunanDomino Dominoes
    kocokKartu(&himpunanDomino)

    fmt.Println("\nKartu domino telah dikocok. Total kartu:",
himpunanDomino.sisaKartu)

    var suitDicari int
    fmt.Print("\nMasukkan suit (0-6) sisi A kartu pembanding
untuk digali: ")
    fmt.Scan(&suitDicari)

    var suitDicariB int
    fmt.Print("Masukkan suit (0-6) sisi B kartu pembanding
untuk digali: ")
    fmt.Scan(&suitDicariB)

    var kartuPembanding Domino
    kartuPembanding.suitA = suitDicari
    kartuPembanding.suitB = suitDicariB
    kartuPembanding.nilai = suitDicari + suitDicariB
    if suitDicari == suitDicariB {
        kartuPembanding.balak = true
    } else {
        kartuPembanding.balak = false
    }

    var kartuHasilGali Domino
    var berhasil bool
    berhasil = galiKartu(&himpunanDomino, kartuPembanding,
&kartuHasilGali)

    if berhasil {
        fmt.Println("\nKartu ditemukan saat menggali
tumpukan:")
        fmt.Println("  Suit sisi A :", kartuHasilGali.suitA)
        fmt.Println("  Suit sisi B :", kartuHasilGali.suitB)
        fmt.Println("  Nilai kartu :", kartuHasilGali.nilai)
        fmt.Println("Sisa kartu pada tumpukan setelah
digali:", himpunanDomino.sisaKartu)

        var hasilPasangan bool

```



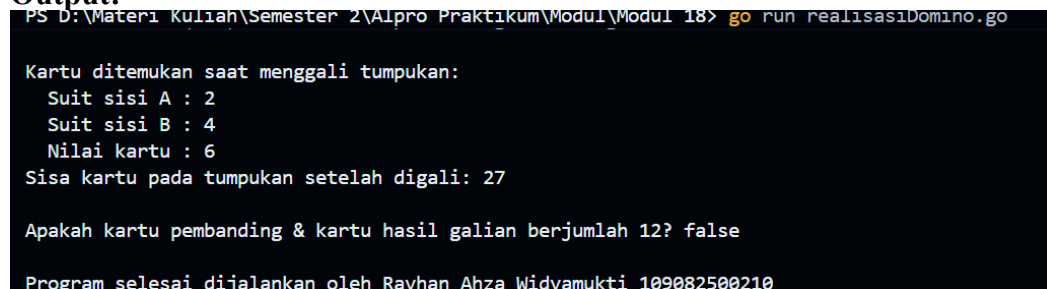
```

        hasilPasangan = sepasangKartu(kartuPembanding,
kartuHasilGali)
        fmt.Println("\nApakah kartu pembanding & kartu hasil
galian berjumlah 12?", hasilPasangan)
    } else {
        fmt.Println("\nKartu dengan suit yang sama tidak
ditemukan, tumpukan habis.")
    }

    fmt.Println("\nProgram selesai dijalankan oleh Rayhan
Ahza Widyamukti_109082500210")
}

```

Output:



```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 18> go run realisasiDomino.go

Kartu ditemukan saat menggali tumpukan:
  Suit sisi A : 2
  Suit sisi B : 4
  Nilai kartu : 6
Sisa kartu pada tumpukan setelah digali: 27

Apakah kartu pembanding & kartu hasil galian berjumlah 12? false

Program selesai dijalankan oleh Rayhan Ahza Widyamukti_109082500210

```

Deskripsi Program:

galiKartu merealisasikan aksi "menggali" tumpukan: ia memanggil ambilKartu berulang kali sampai menemukan kartu yang salah satu suit-nya cocok dengan suit kartu pembanding yang diberikan pengguna, lalu menyimpan hasilnya lewat parameter pointer. sepasangKartu menjumlahkan nilai dua kartu (memakai nilaiKartu) dan mengembalikan true jika totalnya 12. Pengguna memasukkan dua suit untuk membentuk kartu pembanding, lalu program menggali tumpukan dan mengecek apakah hasil galian membentuk pasangan bernilai 12.

3. Implementasi Permainan Gapleh

Source Code:

```
// Rayhan Ahza Widyamukti_109082500210
package main

import "fmt"

type Domino struct {
    suitA int
    suitB int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu    [28]Domino
    sisaKartu int
}

// ----- Operasi dasar mesin domino -----

func kocokKartu(d *Dominoes) {
    var indeks int
    indeks = 0
    var i int
    for i = 0; i <= 6; i++ {
        var j int
        for j = i; j <= 6; j++ {
            d.kartu[indeks].suitA = i
            d.kartu[indeks].suitB = j
            d.kartu[indeks].nilai = i + j
            if i == j {
                d.kartu[indeks].balak = true
            } else {
                d.kartu[indeks].balak = false
            }
            indeks = indeks + 1
        }
    }
    d.sisaKartu = 28

    var kunciAcak int
    kunciAcak = 5
    var k int
    for k = 0; k < 28; k++ {
        var tukar int
        tukar = (k * kunciAcak) % 28
        var temp Domino
        temp = d.kartu[k]
        d.kartu[k] = d.kartu[tukar]
        d.kartu[tukar] = temp
    }
}
```

```

func ambilKartu(d *Dominoes) Domino {
    var kartuTerambil Domino
    if d.sisaKartu > 0 {
        kartuTerambil = d.kartu[d.sisaKartu-1]
        d.sisaKartu = d.sisaKartu - 1
    }
    return kartuTerambil
}

func gambarKartu(kartu Domino, suit int) int {
    var hasil int
    if suit == 0 {
        hasil = kartu.suitA
    } else {
        hasil = kartu.suitB
    }
    return hasil
}

func nilaiKartu(kartu Domino) int {
    var hasil int
    hasil = kartu.nilai
    return hasil
}

func cariKartuSambung(tangan []Domino, ujungKiri int,
    ujungKanan int) (int, string, bool, bool) {
    var indeksTerpilih int
    var sisiTerpilih string
    var perluBalik bool
    var ditemukan bool

    indeksTerpilih = -1
    sisiTerpilih = ""
    perluBalik = false
    ditemukan = false

    var i int
    for i = 0; i < len(tangan); i++ {
        var sA int
        var sB int
        sA = gambarKartu(tangan[i], 0)
        sB = gambarKartu(tangan[i], 1)

        if sA == ujungKanan {
            indeksTerpilih = i
            sisiTerpilih = "kanan"
            perluBalik = false
            ditemukan = true
            break
        }
        if sB == ujungKanan {
            indeksTerpilih = i
            sisiTerpilih = "kanan"
            perluBalik = true
        }
    }
}

```

```

        ditemukan = true
        break
    }
    if sB == ujungKiri {
        indeksTerpilih = i
        sisiTerpilih = "kiri"
        perluBalik = false
        ditemukan = true
        break
    }
    if sA == ujungKiri {
        indeksTerpilih = i
        sisiTerpilih = "kiri"
        perluBalik = true
        ditemukan = true
        break
    }
}
return indeksTerpilih, sisiTerpilih, perluBalik,
ditemukan
}

// hapusDariTangan menghapus kartu pada indeks tertentu dari
slice tangan.
func hapusDariTangan(tangan []Domino, indeks int) []Domino {
    var tanganBaru []Domino
    var i int
    for i = 0; i < len(tangan); i++ {
        if i != indeks {
            tanganBaru = append(tanganBaru, tangan[i])
        }
    }
    return tanganBaru
}

func main() {
    fmt.Println(" SOAL 3 - IMPLEMENTASI PERMAINAN GAPLEH")
    fmt.Println(" Disusun oleh: Rayhan Ahza
Widyamukti_109082500210")

    var himpunanDomino Dominoes
    kocokKartu(&himpunanDomino)

    var jumlahKartuPerPemain int
    fmt.Print("\nMasukkan jumlah kartu yang dibagikan untuk
setiap pemain (misal 5): ")
    fmt.Scan(&jumlahKartuPerPemain)

    var tanganP1 []Domino
    var tanganP2 []Domino

    var i int
    for i = 0; i < jumlahKartuPerPemain; i++ {
        tanganP1 = append(tanganP1,
ambilKartu(&himpunanDomino))
    }

```

```

        for i = 0; i < jumlahKartuPerPemain; i++ {
            tanganP2 = append(tanganP2,
ambilKartu(&himpunanDomino))
        }

        fmt.Println("\nKartu Pemain 1 (P1):")
        for i = 0; i < len(tanganP1); i++ {
            fmt.Println("  [" , i , "] (", tanganP1[i].suitA, ", ",
tanganP1[i].suitB, ") nilai:", tanganP1[i].nilai)
        }
        fmt.Println("Kartu Pemain 2 (P2):")
        for i = 0; i < len(tanganP2); i++ {
            fmt.Println("  [" , i , "] (", tanganP2[i].suitA, ", ",
tanganP2[i].suitB, ") nilai:", tanganP2[i].nilai)
        }
        fmt.Println("Sisa kartu di tumpukan stok:",
himpunanDomino.sisaKartu)

        var indeksPembuka int
        fmt.Print("\nMasukkan indeks kartu P1 yang akan dipakai
untuk membuka rangkaian: ")
        fmt.Scan(&indeksPembuka)

        var rangkaian []Domino
        rangkaian = append(rangkaian, tanganP1[indeksPembuka])
        tanganP1 = hapusDariTangan(tanganP1, indeksPembuka)

        var ujungKiri int
        var ujungKanan int
        ujungKiri = gambarKartu(rangkaian[0], 0)
        ujungKanan = gambarKartu(rangkaian[0], 1)

        fmt.Println("\nRangkaian dibuka dengan kartu (",
ujungKiri, ", ", ujungKanan, ")")

        var giliran int
        giliran = 2 // setelah P1 membuka, giliran berikutnya P2

        var pemainSkip int
        pemainSkip = 0

        for len(tanganP1) > 0 && len(tanganP2) > 0 {
            var tanganAktif []Domino
            if giliran == 1 {
                tanganAktif = tanganP1
            } else {
                tanganAktif = tanganP2
            }

            var idx int
            var sisi string
            var balik bool
            var dapat bool
            idx, sisi, balik, dapat =
cariKartuSambung(tanganAktif, ujungKiri, ujungKanan)

```

```

        if dapat {
            var kartuSambung Domino
            kartuSambung = tanganAktif[idx]

            if sisi == "kanan" {
                if balik {
                    var temp int
                    temp = kartuSambung.suitA
                    kartuSambung.suitA =
kartuSambung.suitB
                    kartuSambung.suitB = temp
                }
                rangkaian = append(rangkaian,
kartuSambung)
                ujungKanan = kartuSambung.suitB
            } else {
                if balik {
                    var temp int
                    temp = kartuSambung.suitA
                    kartuSambung.suitA =
kartuSambung.suitB
                    kartuSambung.suitB = temp
                }
                var rangkaianBaru []Domino
                rangkaianBaru = append(rangkaianBaru,
kartuSambung)
                rangkaianBaru = append(rangkaianBaru,
rangkaian...)
                rangkaian = rangkaianBaru
                ujungKiri = kartuSambung.suitA
            }

            if giliran == 1 {
                tanganP1 = hapusDariTangan(tanganP1, idx)
            } else {
                tanganP2 = hapusDariTangan(tanganP2, idx)
            }

            fmt.Println("\nPemain", giliran, "menyambung
kartu (", kartuSambung.suitA, ", ", kartuSambung.suitB, ") di
sisi", sisi)
            fmt.Println("Rangkaian sekarang: ujung kiri
=", ujungKiri, ", ujung kanan =", ujungKanan)
            pemainSkip = 0
        } else {
            fmt.Println("\nPemain", giliran, "tidak punya
kartu yang cocok, lewati giliran.")
            pemainSkip = pemainSkip + 1
            if pemainSkip >= 2 {
                fmt.Println("\nKedua pemain sama-sama
tidak bisa menyambung. Permainan berhenti (buntu).")
                break
            }
        }

        if giliran == 1 {

```

```

        giliran = 2
    } else {
        giliran = 1
    }
}

if len(tanganP1) == 0 {
    fmt.Println("\nPemain 1 (P1) menghabiskan seluruh
kartunya. P1 MENANG!")
} else if len(tanganP2) == 0 {
    fmt.Println("\nPemain 2 (P2) menghabiskan seluruh
kartunya. P2 MENANG!")
} else {
    var totalP1 int
    var totalP2 int
    for i = 0; i < len(tanganP1); i++ {
        totalP1 = totalP1 + nilaiKartu(tanganP1[i])
    }
    for i = 0; i < len(tanganP2); i++ {
        totalP2 = totalP2 + nilaiKartu(tanganP2[i])
    }
    fmt.Println("\nSisa nilai kartu P1:", totalP1)
    fmt.Println("Sisa nilai kartu P2:", totalP2)
    if totalP1 < totalP2 {
        fmt.Println("Permainan buntu, P1 MENANG (sisa
nilai kartu lebih kecil)!")
    } else if totalP2 < totalP1 {
        fmt.Println("Permainan buntu, P2 MENANG (sisa
nilai kartu lebih kecil)!")
    } else {
        fmt.Println("Permainan buntu dan SERI!")
    }
}

    fmt.Println("\nProgram selesai dijalankan oleh Rayhan
Ahza Widyamukti_109082500210")
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 18> go run gapleh.go

Pemain 2 tidak punya kartu yang cocok, lewati giliran.

Kedua pemain sama-sama tidak bisa menyambung. Permainan berhenti (buntu).

Sisa nilai kartu P1: 18
Sisa nilai kartu P2: 32
Permainan buntu, P1 MENANG (sisa nilai kartu lebih kecil)!

Program selesai dijalankan oleh Rayhan Ahza Widyamukti_109082500210

```

Deskripsi Program:

Gapleh adalah permainan menyambung kartu berdasarkan kecocokan suit di ujung rangkaian. Program ini membagikan sejumlah kartu ke 2 pemain dari tumpukan yang sudah dikocok, lalu pemain bergiliran mencari kartu di tangannya (`cariKartuSambung`) yang cocok dengan ujung kiri atau kanan rangkaian, membalik kartu bila perlu, dan menyambungkannya. Jika kedua pemain berturut-turut tidak punya kartu cocok, permainan dinyatakan buntu dan pemenang ditentukan dari sisa nilai kartu paling kecil; jika salah satu tangan habis lebih dulu, pemain itu menang.

4. Mesin Abstrak Karakter

Source Code:

```
// Rayhan Ahza Widyamukti_109082500210
package main

import "fmt"

type MesinKarakter struct {
    teks      string
    posisi    int
    panjang   int
}

func start(m *MesinKarakter, teksMasukan string) {
    m.teks = teksMasukan
    m.posisi = 0
    m.panjang = len(teksMasukan)
}

func maju(m *MesinKarakter) {
    m.posisi = m.posisi + 1
}

func eop(m *MesinKarakter) bool {
    var hasil bool
    if m.posisi >= m.panjang {
        hasil = true
    } else if m.teks[m.posisi] == '.' {
        hasil = true
    } else {
        hasil = false
    }
    return hasil
}

func cc(m *MesinKarakter) byte {
    var hasil byte
    hasil = m.teks[m.posisi]
    return hasil
}

func main() {
    fmt.Println(" SOAL 4 - MESIN ABSTRAK KARAKTER")
    fmt.Println(" Disusun oleh: Rayhan Ahza Widyamukti_109082500210")

    var teksMasukan string
    fmt.Print("\nMasukkan untaian karakter tanpa spasi (akhiri dengan tanda titik '.'): ")
    fmt.Scan(&teksMasukan)

    var mesin MesinKarakter
```

```

start(&mesin, teksMasukan)
fmt.Println("\n--- Membaca seluruh karakter ---")
for eop(&mesin) == false {
    var karakterSaatIni byte
    karakterSaatIni = cc(&mesin)
    fmt.Printf("%c", karakterSaatIni)
    maju(&mesin)
}
fmt.Println()

start(&mesin, teksMasukan)
var jumlahKarakter int
jumlahKarakter = 0
for eop(&mesin) == false {
    jumlahKarakter = jumlahKarakter + 1
    maju(&mesin)
}
fmt.Println("\nJumlah karakter yang terbaca (tidak
termasuk titik):", jumlahKarakter)

start(&mesin, teksMasukan)
var jumlahHurufA int
jumlahHurufA = 0
for eop(&mesin) == false {
    var karakterSaatIni byte
    karakterSaatIni = cc(&mesin)
    if karakterSaatIni == 'A' {
        jumlahHurufA = jumlahHurufA + 1
    }
    maju(&mesin)
}
fmt.Println("Jumlah huruf 'A' yang terbaca:",
jumlahHurufA)

var frekuensiA float64
if jumlahKarakter > 0 {
    frekuensiA = float64(jumlahHurufA) /
float64(jumlahKarakter)
} else {
    frekuensiA = 0
}
fmt.Printf("Frekuensi kemunculan huruf 'A' terhadap
seluruh karakter: %.4f\n", frekuensiA)

start(&mesin, teksMasukan)
var jumlahPasanganLE int
jumlahPasanganLE = 0
var karakterSebelumnya byte
karakterSebelumnya = 0
for eop(&mesin) == false {
    var karakterSaatIni byte
    karakterSaatIni = cc(&mesin)
    if karakterSebelumnya == 'L' && karakterSaatIni ==
'E' {
        jumlahPasanganLE = jumlahPasanganLE + 1
    }
}

```

```

        karakterSebelumnya = karakterSaatIni
        maju(&mesin)
    }
    fmt.Println("Jumlah pasangan huruf 'L' diikuti 'E'
(\"LE\") yang terbaca:", jumlahPasanganLE)

    fmt.Println("\nProgram selesai dijalankan oleh Rayhan
Ahza Widyamukti_109082500210")
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 18> go run mesinKarakter.go

--- Membaca seluruh karakter ---
ALELALELELE

Jumlah karakter yang terbaca (tidak termasuk titik): 11
Jumlah huruf 'A' yang terbaca: 2
Frekuensi kemunculan huruf 'A' terhadap seluruh karakter: 0.1818
Jumlah pasangan huruf 'L' diikuti 'E' ("LE") yang terbaca: 4

Program selesai dijalankan oleh Rayhan Ahza Widyamukti_109082500210

```

Deskripsi Program:

Mesin ini punya 4 operasi dasar: start (menyiapkan posisi baca di awal teks), maju (memajukan posisi), eop (mengecek apakah sudah sampai titik akhir), dan cc (membaca karakter di posisi sekarang). Dengan keempatnya, program membangun lima algoritma: membaca seluruh karakter, menghitung jumlah karakter terbaca, menghitung jumlah huruf "A", menghitung frekuensi huruf "A" terhadap total karakter, dan menghitung jumlah pasangan huruf "L" diikuti "E". Karena fmt.Scan membaca token sampai whitespace, input harus berupa untaian tanpa spasi yang diakhiri titik.

DAFTAR PUSTAKA

- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Liskov, B., & Guttag, J. (2000). *Program Development in Java: Abstraction, Specification, and Object-Oriented Design*. Addison-Wesley.
- Munir, R. (2016). *Algoritma dan Pemrograman dalam Bahasa Pascal, C, dan C++*. Informatika Bandung.
- Tim Penyusun Modul Praktikum. (2024). *Modul Praktikum Algoritma dan Pemrograman 2*. Fakultas Informatika, Telkom University.
- Hoare, C. A. R. (1972). Proof of correctness of data representations. *Acta Informatica*, 1(4), 271–281.
- Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). *Data Structures and Algorithms*. Addison-Wesley.